

环境监测项目 · 第二版开发过程与问题整理

本版目标：把温度显示到 LCD1602A 上（串口打印保留）**硬件：**普中 A2 开发板（STC89C52RC, 11.0592MHz）+ LCD1602A 字符屏 + DS18B20 **开发环境：**Keil uVision + VS Code **记录时间：**2026-10-04 **结果：**✅ 屏幕正常显示温度，串口同步输出

一、本版成果

项目	状态
LCD1602A 驱动（7 个函数）	✅ 自己敲完
屏幕显示温度	✅ 实机验证通过
串口同步打印温度	✅ 保留
代码总量	3 个 .c (main / lcd1602 / public) + 2 个 .h

屏上效果：

TEMP: 25

二、硬件与接口

2.1 LCD1602A 关键参数

项目	值
类型	字符型液晶（不是图形屏）
容量	16 列 × 2 行 = 32 个字符
控制器	HD44780 兼容
工作电压	5V
对比度	V0/VO 脚接电位器调节
接口	并行（8 位或 4 位）

2.2 引脚定义（普中 A2 板）

```
sbit LCD1602_RS = P2^6;           // 数据/命令选择
sbit LCD1602_RW = P2^5;           // 读写选择
sbit LCD1602_E  = P2^7;           // 使能
#define LCD1602_DATAPORT P0        // 8 位数据口 = P0 整口
```

2.3 并行 vs 串行 (和 LCD9648 的差别)

项目	LCD1602 (并行)	LCD9648 (串行 S8)
数据线	8 根 (P0 整口)	1 根 (SDA)
送一个字节	1 步	8 步
数据被收走的时刻	E 的下降沿	SCL 的上升沿
片选	无 (E 兼)	CS 专用
复位脚	无	RST
占用 IO	11 根	5 根

术语澄清: LCD9648 的 S8 属于「4 线 SPI」 (手册原文), SPI 全称 Serial Peripheral Interface, 本身就是串行的一种。但中文里「串口」默认指 UART, 所以**不要用「串口」称呼 SPI**。

三、驱动架构 (7 个函数)

分层设计, 上层调下层:

第 1 层	底层收发	write_cmd / write_data
第 2 层	初始化	init / clear
第 3 层	定位	set_cursor
第 4 层	显示	show_char / show_string / show_num

3.1 写命令 / 写数据 (时序 8 步)

步	动作
1	RS 设好 (0 = 命令, 1 = 数据)
2	RW = 0 (写模式)
3	E = 0
4	数据放上 P0 (8 位)
5	延时 (等稳定)
6	E = 1
7	延时
8	E = 0 ← 下降沿, 屏在这一刻把 8 位收进去

3.2 初始化 (4 条命令)

顺序	命令	作用
1	0x38	8 位总线、2 行、5×8 点阵
2	0x0C	开显示、无光标、不闪烁
3	0x06	写入后光标右移、屏幕不移动

顺序	命令	作用
4	0x01	清屏

(前面还有 delay_ms(20) 等上电稳定。)

3.3 各函数职责

函数	干什么
lcd1602_write_cmd(u8 cmd)	发一条命令
lcd1602_write_data(u8 dat)	发一个字符
lcd1602_init(void)	开机初始化
lcd1602_clear(void)	清屏
lcd1602_set_cursor(u8 x, u8 y)	把光标挪到 (x, y)
lcd1602_show_char(u8 x, u8 y, u8 ch)	指定位置显示一个字符
lcd1602_show_string(u8 x, u8 y, u8 *str)	显示一串文字
lcd1602_show_num(u8 x, u8 y, u16 num)	指定位置显示数字

四、核心概念：光标与内部地址

4.1 光标是什么

屏内部有个「光标」（地址计数器）。规则：

- 你写数据时，数据落在**光标当前位置**
- 写完一个，**光标自动右移一格**（因为初始化发了 0x06）

所以"显示字符"永远是两件事：① 把光标挪过去 ② 把字符写进去。

⚠️ 光标在屏里，你的程序里没有这个变量 —— 只能通过"发命令"去改它。

4.2 内部地址布局

	列0	列1	...	列15		列16	...	列39
第 1 行：	0x00	0x01	...	0x0F		0x10	...	0x27
第 2 行：	0x40	0x41	...	0x4F		0x50	...	0x67
	└─ 看得见 ─┘					└─ 看不见 ─┘		

两个要点：

- 第 2 行从 0x40 开始（不是 0x10）—— 因为 HD44780 每行预留 40 个格子，兼容各种尺寸的屏
- 右边那些有地址、但屏上没有像素 —— 写进去也看不见

4.3 地址换算

```
y = 0 (第 0 行): 地址 = 0x00 + x
y = 1 (第 1 行): 地址 = 0x40 + x
```

为什么判断 **y** 不判断 **x**: **x** 的范围两行一样 (0~15), 不改变基址; 只有 **y** 决定用 0x00 还是 0x40。

x 往右数 (列), **y** 往下数 (行), 都从 0 起, (0,0) 在左上角。

4.4 0x80 是什么

它是"设置光标地址"这条命令的固定格式, 不是通用的"命令前缀"。

命令门里不同数值 = 不同命令:

命令值	作用
0x01	清屏
0x06	输入模式
0x0C	开显示
0x38	功能设定
0x80 + 地址	设置光标位置 (最高位是 1 表示"后面是地址")

只有"设置地址"这一条要 0x80, 其他命令和所有数据都不加。

4.5 命令 vs 数据: 两道门

靠 RS 引脚分开:

RS	进哪道门	字节被当成
0	命令门	指令
1	数据门	要显示的字符

为什么数据不需要任何"前缀":

- 命令门里东西多 (清屏、设地址、开显示...) → 屏需要知道"是哪种" → 靠数值编码
- 数据门里只有一种东西: 字符 → 没什么可区分的 → 直接给字符

同样一个 0x41: 走命令门 = 无效指令 (忽略); 走数据门 = 显示 'A'。

4.6 ⚠ 屏不会自动换行

地址只会 +1。写到 0x0F 后再写一个, 会落到 **0x10** —— 屏认为"还在第 1 行的第 17 格", 但那里没有像素, 字符消失。

要换行必须手动发命令跳到 0x40 区。官方 `lcd1602_show_string` 里那段 `if(i < 16-x) ... else ...` 就是在做手动换行。

五、写代码踩的坑（按发现顺序）

坑 1: `delay_10_us` 名字多了一个下划线

写错了	正确
<code>delay_10_us</code>	<code>delay_10us</code>

`public.h` 里声明的是 `delay_10us`（10 和 us 之间无下划线）。一字之差 → 报"未定义"。

教训：函数名以头文件里的声明为准，不能凭印象写。

坑 2: `write_data` 里变量名写错

```
void lcd1602_write_data(u8 dat)
{
    LCD1602_DATAPORT = cmd;    // ✗ 参数叫 dat，不是 cmd
}
```

原因：从 `write_cmd` 复制过来忘了改。

坑 3: 引脚名两边对不上

文件	写的
<code>lcd1602.h</code>	<code>LCD1602_RS / LCD1602_RW / LCD1602_E</code>
<code>lcd1602.c</code>	<code>RS / RW / E</code>

→ 编译报 `undefined identifier`。

规则：头文件里怎么定义，源文件里就得怎么写，一字不差。

坑 4: 在 `show_string` 里重复实现 `set_cursor` 的活

```
void lcd1602_show_string(u8 x, u8 y, u8 *str)
{
    if(y == 0)                                // ✗ 这是 set_cursor 已经干过的
        lcd1602_write_cmd(0x80 + x);
}
```

应该直接调：

```
lcd1602_set_cursor(x, y);    // ✔ 一行搞定
```

教训：上层函数调用下层函数，不要重复造轮子。

坑 5: `'x'` 和 `x` 是两回事

写法	含义
<code>x</code>	变量（调用时传进来的列号）

写法	含义
'x'	字符常量，值是字母 x 的 ASCII 码 = 0x78 = 120

写成 `0x80 + 'x'` 会算出 `0x80 + 120 = 0xF8`，完全不是想表达的位置。

记住：有引号 = 字符本身；没引号 = 变量。

坑 6：函数名拼错 + 参数没写名字

```
void lcd1602_shou_string(u8 x, u8 y, u8 )    // ❌
```

- `shou` → 应该是 `show`
- 第三个参数只有类型 `u8`，**没有名字** → 编译报错
- 正确：`u8 *str`

教训：函数名和参数，都要跟头文件里的声明**完全一致**。

坑 7：少定义了全局变量

`ReadByte()` 里用了 `dat`，但 `main.c` 里没有定义它。

原因：从旧文件复制时漏了这一行。补上 `u8 dat = 0;` 即可。

坑 8：main 函数嵌套了

```
void main()//主程序
{
    void main(void)    // ❌ C 语言不允许函数套函数
    {
        u8 t;
        ...
    }
}
```

原因：复制代码时，旧的 `main` 头和新写的 `main` 头都留着了。

改法：删掉旧的那两行，只留一个函数头。

坑 9：函数定义和声明的参数类型不一致

文件	写的
lcd1602.h	<code>void lcd1602_show_num(u8 x, u8 y, u16 num);</code>
lcd1602.c	<code>void lcd1602_show_num(u8 x, u8 y, u8 num)</code>

改成统一 `u16`（好处：以后温度上 3 位数也装得下）。

六、C 语言语法坑

坑 10: 'T:' 是非法字符常量

写法	含义	能不能用
'T'	单个字符	✓
':'	单个字符	✓
'T:'	单引号里 2 个字符	✗ 非法
"T:"	双引号 = 字符串	✓ (但那是 show_string 用的)

规则：单引号里只能放一个字符。

所以"T:" 要这么写：

```
lcd1602_show_char(0, 0, 'T');  
lcd1602_show_char(1, 0, ':');
```

坑 11: C 语言里不能 "文字" + 变量

```
lcd1602_show_string(0, 0, 'Temp :' + t);    // ✗ 三重错误
```

1. 用了**单引号** (应该是双引号)
2. **C 语言里 + 不能拼字符串** (Python/Java 可以, C 不行) —— "Temp :" + t 实际是"把字符串地址往后挪 t 格", 指向垃圾数据
3. 数字 t 也不会自动变成字符

正确做法：分两步

```
/* 固定文字：一个个 show_char */  
lcd1602_show_char(0, 0, 'T');  
lcd1602_show_char(1, 0, 'e');  
lcd1602_show_char(2, 0, 'm');  
lcd1602_show_char(3, 0, 'p');  
lcd1602_show_char(4, 0, ' ');  
lcd1602_show_char(5, 0, ':');  
  
/* 数字：用 show_num */  
lcd1602_show_num(7, 0, t);
```

坑 12: 为什么要 + '0'

```
lcd1602_write_data(num / 10 + '0');    // 这个 + '0' 不能省！
```

因为数值和字符在计算机里是两回事：

你写的	内存里的值	屏上显示
数值 2	2	一个看不见的控制字符
字符 '2'	50	显示 2

字符 '0' 的 ASCII 码是 48，而且 '0' ~ '9' 是连续排列的：

```
'0' = 48    '1' = 49    '2' = 50    ...    '9' = 57
```

所以 数值 + 48 = 对应字符，写 + '0' 比写 + 48 更好懂。

一句话：屏只认字符，不认数值。数字要显示出来，必须先 + '0'。

坑 13：指针看不懂 → 改用数组下标

show_string 里官方用的是指针写法：

```
while(*str != '\0') { write_data(*str); str++; }
```

看不懂的话，改成下标写法，效果完全一样：

```
u8 i;
for(i = 0; str[i] != '\0'; i++)
{
    lcd1602_write_data(str[i]);
}
```

指针写法	下标写法	意思
*str	str[i]	当前字符
str++	i++	往后挪一格

参数还是要写 u8 *str（C 语言传字符串就必须这么写，跟 u8 str[] 等价），但函数体里可以当数组用。

七、Keil 工程配置

7.1 工程里要加哪些文件

只加 .c，.h 不用加（编译器会顺着 Include Paths 自己找）：

```
App\main.c
App\lcd1602.c
Public\public.c
```

7.2 Include Paths 必须配

Project → Options for Target → C51 → Include Paths，加两条：

App
Public

⚠ **每加一条要点一次 Add** —— 只选文件夹不点 Add，路径不会进列表。

不配的后果：报 `can't open file 'xxx.h'`。

7.3 必须勾上 Create HEX File

Options for Target → Output → **勾 Create HEX File** —— 不勾就没有 hex，烧不进板子。

7.4 Keil 的「组 (Group)」≠ 磁盘文件夹

- **磁盘文件夹** = 文件真实位置
- **Group** = 工程树里的**虚拟分类**，只为好看

Keil 不会自动按磁盘目录分组，得手动建。而且**树最多两层** (Target → 组 → 文件)，不支持深层嵌套。

分组不影响编译，别为它耗时间。

7.5 编译失败先看日志

<工程目录>\Obj\<工程名>.plg

用记事本打开 (前面有 HTML 标签，往下拉就是正文)，里面写清了**哪个文件、哪一行、什么错误**。

7.6 一个高频错误：REGX52.H 和 reg52.h 冲突

REG52.H(13): error C231: 'P0': redefinition
(一大串)

原因：`.c` 里 `#include <REGX52.H>`，而 `public.h` 里 `#include "reg52.h"` —— **两个文件定义同一批寄存器**。

规则：`include` 了 `public.h` 就不要 `include` `REGX52.H`，二选一。

八、调试经验

8.1 字符看不清 → 调对比度

板上 LCD 接口旁边的小**电位器**，慢慢转。

转到一头	现象
某个方向到底	出现 16 个方块 (调过头)
反方向到底	什么都看不见
中间某个位置	字符清晰 ← 停这

技巧：先故意转到出现黑方块，再慢慢往回转，转到方块消失、字符刚清楚。

8.2 每写一个函数就编译一次

别一口气全写完再编译 —— 出错了不知道是哪儿的问题。

推荐顺序：写底层 → 编译 → 加上层 → 编译 → 写 main → 编译。

九、经验总结

1. 函数名、参数名、参数类型，一切以头文件声明为准 —— 拼错一个字符就是"未定义"
2. 上层函数调下层函数，别重复实现（`show_string` 里直接调 `set_cursor`）
3. 单引号只能一个字符，双引号才是字符串
4. C 语言不能 "字符串" + 变量 —— 跟 Python/Java 不一样
5. 数值要显示，必须先 + '0' 转成字符
6. 屏不会自动换行 —— 地址只管 +1，换行要手动设地址
7. 只有"设置地址"这条命令要 `0x80`，其他命令和所有数据都不加
8. RS 决定"命令还是数据"，数值决定"哪种命令/哪个字符" —— 两道门互不干扰
9. 一次只改一个地方，改完立刻编译 —— 同时改多处，出错就不知道是谁的问题
10. Keil 编译失败，先看 `obj\<工程名>.plg`

十、下一步计划

1. 补上「数组和字符串」这一课 —— 学会之后 `show_string` 就能用了，不用一个个 `show_char`
2. 加蜂鸣器越限报警（温度超过阈值就响）
3. 按键设置报警阈值（不用改代码就能调）
4. 非阻塞重构（用定时器驱动，去掉 `delay_ms` 阻塞）
5. 增加显示小数温度（现在只显示整数）

本文档记录第二版从零到屏幕显示成功的完整过程，代码均经实机验证。